1. Demonstrate how a child class can access a protected member of its parent class within the same Package.

⇒ Accessing protected member in same package

parent.java

```
Package.pack1;
public class Parent {
    Protected string message = "Hello from P";
```

child.Java

```
Package pack1;
public class child extends Parent{
  public void showMessage(){
      System.out.println("child accessed :"+message);
    }
   Public static void main(String args[]){
       child c = new child();
         c.showMessag();
   }}
```

Since both classes in the same package the protected member message is accessible in child class.

Protected in different classes

```
Package Pack1;
public class Parent{
    Protected String message="Hello";
  }
```

```
Package pack2;
impont Pack1. Parent;
Public class child extends Parent {
Public void showMessage (){
 System. out. println ("Access from child :" + message);
 }
  public static void main (String [] args){
     child c = new child ();
      c. showMessage ();
    }
   }
```

A sub class in a different package can access protected member of the Parent class only through inheritance, not through the object of the parent.

Lab-1:

2. Compare abstract classes and interfaces in terms of multiple inheritance.

Feature	Abstract class	Interface
multiple inheritance	Not supported	Fully supported
extends	class A extends class B	class A implements X, Y, Z
Code reuse	Can have method bodies and member variable.	Java 8+ can have default and static methods.
state	Can have instance variables.	Only constants (public static final)
Constructor	yes	No constructor allowed.

When to use an abstract class:

(i) ✗ to provide base functionality and shared states.

(ii) need of constructor or non-static instance variables.

(iii) expected closely related classes with an "is-a" relationship.

When to use interface:

(i) Want to define pure behavior, not implementation.

(ii) Want to use multiple inheritence of type.

(iii) classes are unrelated but share common capabilitie

Lab-2:

3] How does encapsulation ensure data security and integrity ? Show with an bank account class using private variables and validated methods such as setAccountNumber (String) etc.

⇒ Encapsulation is a key principle in object-oriented programming that hide internal data. It helps data security, data integrity, Maintainability.

```
public class BankAccount{
    Private String ~~Account~~ accountNumber;
    Private double balance;
public void setAccountNumber (String AccountNumber){
if (accountNumber == null || accountNumben trim(). is Emp
{ throw new IllegalArgumentException ("Can't be null");
}    this. accountNumber = accountNumber;}
Public void setInitialBalance (double ₹ balance){
if (balance<0){
    throw new IllegalArgumentException ("Balance can't
                                        be negative");}
 this.balance =balance;
}
Public String getAccountNumber (){
return accountNumber;
}
```

```
public double getBalance(){
    return balance;}
Public void deposit(double amount){
    if (amount>0) this.balance += amount;
}}
```

4)

i) find kth smallest element:

```
import java.util.*;
Public class kthSmallest{
Public static void main(String[] args){
List<Integer> list = Arrays.asList(8,2,5,1,9,4);
Collection.sort(list);
int k = 3;
System.out.println(k+"rd smallest "+list.get(k-1));}}
```

output: 4

ii) Word freequency using treemap

```
import java.util.*;
Public class Wordfreq{
  Public static void main(String[] args){
 String text = "apple banana apple mango";
TreeMap<String, Integer> map = new TreeMap<>();
for (String word : text.split(" "))
map.put(word, map.getOrDefault(word,0)+1);
map.forEach((K,V) -> System.out.print(K+"="+V));
}}
```

output: apple = 2
banana = 1
mango = 1

(iii) Queue & stack using PriorityQueue.

⇒

```
import java.util.*;
Public class PgstackQueue{
    Static class Element{
    int val, order;
    Element(int v, int o){val=v; order=o;}}

Public static void main(String[] args){
PriorityQueue<Element> stack = new PriorityQueue<>
((a,b)->b.order - a.order);
PriorityQueue<Element> queue = new PriorityQueue<>
(comparator.comparingInt(a->a.order));
int order=0;
stack.add(new Element(10, order++));
stack.add(new Element(20, order++));
System.out.println("Stack pop:" + stack.poll().val);
queue.add(new Element(100,0));
queue.add(new Element(200,1));
System.out.println("Queue poll:" +queue.poll().val);
}}
```

Lab 3 :-
5

Multithread based project

```
import java.util.*;
import java.util.concurrent.*;
class ParkingPool {
  Private final Queue <String> queue = new LinkedlistO();
Public synchronized void addCar(String car){
queu.odd (Car);
System.out.println("Car" +car+" requested park
                    -ing.");
notify ();}
Public synchronized String getCar () {
while (queue.isEmpty ()){
try { wait ();} catch (Interrupted Exception e)
{}
return queue.poll ();
}}
class RegistenParking extends Thread {
  Private final String CarNumber;
  Private final parkingPool pool;
  Public RegistenParking (String CarNumber, Parking Poo
                          Pool){
  this.carNumber = carNumber;
  this.pool = pool;}
  Public void run () {
    Pool.addCar (CarNumber);}}
```

```
class ParkingAgent extends Thread {
private final parkingPool pool;
private final int agentId;
Public ParkingAgent (ParkingPool pool, int
                                agentId){
this.pool = pool;
this.agentId = agentId; }
public void run (){
 while (true) {
    String can = pool.getCan ();
System.out.println ("Agent " + agentId + "
                    Parked can " + can + " ");
~~System.out.println ("Ag~~
try {
    Thread.sleep (500);}
catch (InterruptedException e ){}
 }}
  }
```

```
public class CarParkingSystem {
    Public static void main(String args[]) {
    ParkingPool pool = new ParkingPool();
new ParkingAgent(pool, 1).start();
new ParkingAgent(Pool, 2).start();
new RegisterCarParking("ABC123", pool).start();
new RegisterCarParking("XYZ456", Pool).start();
}}
```

Output:

```
Car ABC123 requested Parking.
Car XYZ456 requested Parking.
Agent 1 Parked car ABC123.
Agent 2 parked car XYZ456.
```

6] Comparison between DOM VS SAX.

Feature	DOM	SAX
Memory	High (loads whole XML)	Low (reads line by line)
Speed	Slower for big files	faster for large fil
Navigation	Easy (tree structure)	Hard
Modification	Yes	NO
Best for	Small XML, editing	Large XML

7. How does the virtual DOM in React improve performance?

⇒ React creates a virtual copy of DOM. on update, React:

(i) Compares (diffs) old and new virtual DOM.

(ii) finds whats changed.

(iii) Applies only the changes to real DOM.

8| Event delegation in JavaScript:

⇒ A technique where a single event Listener is attached to a parent element to handle events from current and future child elements.

```
<ul id = "menu">
<li > Home .</li>
<li> About </li>
</ul>
<script>
document. getElementById ('menu'). addEventListener
                                ('click', function (event){
if (event.target.tagName == 'ME') {
alert ('Clicked :' + event.target.textContent);
}});
</script>
```

9] Explain how Java Regular Expressions can be used for input validation.

⇒ Java Provides the pattern and Matcher classes in the java.util.regex package to work with regular expressions.

```
import java.util.regex.*;
public class EmailValidator {
    public static void main (String [] args):
        String email = "user@example.com";
        String regex = "^[\\w.-]+@[\\w.-]
                       ++\\.[a-zA-Z]{2,6}$";

Pattern pattern = Pattern.compile(regex);
Matcher matcher = pattern.matcher(email);
if (matcher.matches()) {
    System.out.println("Valid email.");
}
else{
    System.out.println("Invalid email.");
}}
```

10. What are custom annotations in Java and how are they used at runtime using reflection?

⇒ Custom annotations allow to define metadata for the Java code.

Defining a custom Annotation:

```
import java.lang.annotation.*;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnnotation {
    String value();
}
```

Using the annotation:

```
public class TEST {
    @MyAnnotation(value = "example")
    public void display() {
        System.out.println("Display method");
    }
}
```

Processing the Annotation with Reflection:

```
import java.lang.reflection.*;
public class AnnotationProcessor {
    public static void main(String[] args)
            throws Exception {
        for (Method method : Test.class.getDeclare-
                -Methods()) {
```

```
if (method.IsAnnotationPresent(MyAnnotation.class)){
    MyAnnotation annotation = method.getAnnotation
                              (MyAnnotation.class);
    System.out.println("Annotation value: " + annotation.value());
}
}
}
```

11| Discuss the Singleton design pattern in Java.

⇒ The ~~sig~~ singleton pattern ensures that only one instance of a class exists and provides a global access point to it.

it solves :-

(i) prevents multiple instances.

(ii) Controls resource usage and ensures consistency.

Basic singleton implementation

```
public class Singleton {
    private static Singleton instance;
    private Singleton(){}
    Public static Singleton getInstance(){
        if (instance == null){
            instance = new Singleton();
        } return instance;
    }
}
```

Thread-safe Singleton:

```
Public class singleton {
 proirate static volatile Singleton instance;
  private Singleton (){}
    public static Singleton getInstance (){
        if (instance ==null){
            Synchronized ( Singleton. class){
              if (instance ==null){
                 instance =new Singleton ();
              }
            }
    return instance.
    }}
```

Lab 4: 12] Describe how JDBC manages communication between a Java application and a relational database.

⇒ JDBC (Java Database Connectivity) is an API that allows Java applications to connect to and interact with relational database like MySQL, PostgreSQL, or oe oracle.

Steps to Execute a SELECT Query:

```
import java.sql.*;
public class DBExample {
   public static void main (String[] args){
```

```
string url = "jdbc:mysql://localhost:3306/mydb";
string user = "root";
string pass = "password";
Connection conn = null;
Statement stmt = null;
ResultSet rs = null;

try {
 Conn= DriverManager.getConnection(url, user, pass);
  stmt = conn.createStatement();
   rs = stmt.executeQuery("Select * from employee

while (rs.next()) {
    System.out.println("Name: " + rs.getString("Name
  } catch(SQLException e){
  System.out.println("Error: " + e.getMessage());
    } Finally {
     try{
      if (rs != null) rs.close();
      if (stmt != null) stmt.close();
      if (conn != null) conn.close();
     } catch (SQLExeception ex){
        ex.printStackTrace();
    }
```

13] How do servlets and JSPs work together in a web application following the MVC (model-view-Controller) architecture? Provide a brief use case showing the servlet as a controller, JSP as a view, and a Java class as the model.

⇒ Servlet acts as the Controller, JSP is the view and Java class is the Model in a Java EE web application using the MVC architecture.

How they work together:

① Client sends an HTTP request to the servlet.

② Servlet processes the request:

③ JSP retrives the data and renders the HTML response to the client.

Sample Use case: Displaying User Profile

Model: User.Java

```
Public class User{
    Private string name;
    Private String email;
Public User(String name, String email){
    this.name = name;
}   this.email = email;
```

```
Public String getName () { return name; }
Public String getEmail () { return email; }
}
```

Controller : UserServlet.Java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class UserServlet extends HttpServlet{
    protected void doGet (HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException{
    User user = new User ("Suparna Paul", "Sup@example.com");
        request.setAttribute ("user", user);
RequestDispatcher dispatcher = request.getRequestDispatcher
                                    (Profile.jsp);
        dispatcher.forward (request, response);
    }
}
```

Profile.jsp

```
<%@ page import = "yourpackage.User" %>
<%
    User user = (User) request.getAttribute("user");
%>
<html>
<head><title> User profile </title></head>
<body>
 <h2> User Profile</h2>
 <P> Name : <% =user.getName() %> </P>
 <P> Email : <% =user.getEmail() %></P>
 </body>
 </html>
```

14] Explain the life cycle of a Java servlet. What are the roles of the init(), service(), and destroy() methods? Discuss how servlets handle concurrent ge requested and how thread safety issues may ~~anni~~ arise.

⇒ The life cycle of a Java servlet is managed by the Servlet Container. It involves three main stages, handled by three important methods:

1. init() Method

- The init() method is called once when the servlet is first loaded into memory.
- It is used to initialize resources, such as database connections or configure settings.

2. Service() Method

- The service() method is called each time a client makes a request.
- It determines the HTTP request type (Get, POST etc) and calls the appropriate method.
- This is the where the main request handing logic goes

3. Destroy() method:

- The destroy() method is called once when the servlet is being removed from memory.
- It is used to release resources, such as closing database connections.

Handling Concurrent Requests:

- A servlet is single instance and multithreaded.
- This means the servlet container creates only one instance of the servlet, but can handle multiple requests [illegible] different threads.

Thread Safety Issues:

Since multiple threads share the same servlet instance, shared variables can lead to race conditions.

For example:

```
private int counter = 0;
protected void doGet(...){
    counter++;
}
```

[5] A single instance of a servlet handles multiple requests using threads. What problems can occur if shared resources are accessed by multiple threads? Illustrate your answer with an example and suggest a solution using synchronization.

⇒ In a servlet, the container creates only one instance, and multiple requests are handled using separate threads. If shared resources are accessed or modified by these threads simultaneously, it can lead to thread safety issues, like:

- Race Conditions
- Incorrect data
- Unpredictable behavior

Example: ~~Race Condition~~ Suppose we have a servlet that ~~occurs~~ counts how many users visited a page

```
public class CounterServlet extends HttpServlet{
  private int counter=0;
 Protected void doGet(HttpServletRequest req,
                      HttpServletResponse res)
   throws ServletException, IOException{
       counter++;
      res.getWriter().println("Visitor number:"
                              +counter);
}}
```

Solution using synchronization:

```
public class CounterServlet extends HttpServlet{
  private int counter=0;
Protected void doGet(HTTPServletRequest req,
                     HTTPServletResponse res)
   throws ServletException, IOException{
       synchronized(this){
       counter++;
       res.getWriter().println("Visitor number:"
                               +counter);
}}}
```

The synchronized block ensures that only one thread can access the counter at a time.

This prevents race conditions and ensures data consistency.

16] Describe how the MVC pattern separates concerns in a Java Web application. Explain the advantages of this structure in terms of maintainability and scalability. Using a student reg system as an example.

⇒ The Model-View-Controller pattern is a design architecture that separates concerns in a Java web application. It divides the application into three components:

① Model (M):
- Represents the data and business Logic.
- Contains Java classes.
- Handles database interactions, validations and computations;

② View (V):
- Represents the presentation layer.
- Implemented using JSPs, HTML, CSS
- Displays data received from the controller but

does not handle logic.

(iii) Controller (C):

- Acts as the intermediary between Model and View.
- Implemented using Servlets.
- Handles requests, processes input, updates model and selects the appropriate View.

Example: Student Registration System

- View: (JSP): register.jsp – displays a registration form to the user.
- Controller (servlet): RegisterServlet.java – receives the form data, calls the model, and forwards to use a success/failure page.
- Model (Java class) :- Student.java, StudentDAO.java - handles business logic and stores student data into the database.

Advantages:

(i) Separation of Concerns: Each component has a distinct role, making the codebase clean and organized.

(ii) Maintainability: Changes in UI don't affect business logic and vice versa.

(III) Reusability: Same model logic can be reused in different views.

(IV) ~~Seab~~ Scalability: Easier to scale and add new features.

(V) Testability: Logic is isolated in the model, which makes # unit testing easies.

~~Lab 5:~~

~~Lab 5 - 17:~~

Lab 5:

17| In a Java EE application, how does a servlet controller manage the flow between the model and the view? Provide a brief example that demonstrates forwarding data from a servlet to a JSP and rendering a response.

⇒ How servlet Controller Manages Flow (MVC):-

In a Java EE web app, the Servlet acts as a controller.

It:

- Takes user input (request).
- Uses the model (Java class) to process data.
- Then forwards data to JSP (view) to show results.

Sample Example:

1) Model (User.java)

```
public class User {
    private string name, email;
    public User (String name, String email){
        this.name = name; this.email = email;
    }
public String getName () { return name; }
public String getEmail () { return email; }
}
```

ii) Controllen (UserControllen.java)

```
@WebServlet ("/user")
public class UserControllen extends HttpServlet {
protected void doPost(HttpServletRequest req,
                      HttpServletResponse res)
throws ServletException, IOException {

    String name = req.getParameter ("name");
    String email = req.getParameter ("email");
     User user = new User(name,email);
     req.setAttribute ("user", user);
RequestDispatcher rd = req.getRequestDispatcher
                                ("userInfo.jsp");
     rd.forward (req, res);
      }
      }
```

iii) View (userInfo.jsp)

```
⇒ <%@ page import = "model.User" %>
  <%
    User user = (User) request.getAttribute("user"
  %>
  <h2> User Info </h2>
  <P> Name : <%= user.getName() %> </P>
  <P> Email : <%= user.getEmail() %> </P>
```

18] Compare and Contrast cookies, URL rewriting, and HttpSession as methods for session tracking in Servlets. Discuss their advantages, limitations and idel use cases.

Aspect	Cookies	URL Rewritting	HttpSession
1. Where data is stored	On client browser	In the URL as query parameters.	On the Server.
2. Visibility to user	Yes	Yes	No
3. Security level	Low	Low	High
4. Data Limit	Limited (4KB per cookie)	Limited (due to URL Length	Large (depends on server memo
5. Implement-ation comple-xity	Medium	High	Low

Advantages, Limitations & Ideal Use Cases:

Cookies

Advantages: Can persist data even after browser is close and Good for user preference.

Limitations:- Can be disabled by user. Low security - data is visible & modifiable.

Ideal use cases: Remembering usernames or preferences. Tracking returning users

URL rewriting

Advantages: Works when cookies are disables. Easy to implement for small apps or links.

Limitations: Exposes data in URL (security risk) Must rewrite every URL manually.

Ideal use cases: Temporary session tracking when cookies are turned off. Basic apps without login.

HttpSession:

advantages: secured and easy to use with built-in support in java. Automatically expires after timeout.

Limitations: Uses server memory. Needs cleanup to avoid memory leaks in large apps.

Ideal Use cases: Login systems, shopping carts, personal dashboards.

19) A web application stores user login information using HttpSession. Explain how the session works accross multiple requests and how session timeout or invalidation is handled securely.

⇒ In a web application, HttpSession is used to store user-specific data, such as login information, accross multiple requests. When a user logs in, the server creates a session object using request.getSession() and stores attributes like username or user ID in that session.

Each user is assaigned a unique session ID, which is typically stored in a cookie (JSESSIONID) on the ~~browse~~ user's browser. This ID is sent automatically with each request, allowing the server to recognize the user and retriever their session data

- Working accross Multiple Requests:
  - When a user logs in, the server creates a session and gives a unique session ID. This session ID is sent with every request. The server uses the ID to find the session and show user-spef specific data.
- Session Timeout and Invalidation: If the user is inactive, the session ends automatically. The session can also be ended using session.invalidate(). After timeout or logout, the user must log in again.

- Security Handling: User data is kept on the server, not on the browser. Session ID should be sent over HTTPs to avoid hacking.

20] Explain how Spring MVC handles an HTTP request from a browser. Describe the role of the @Controller, @RequestMapping, and model objects in separating business logic from presentation. Provide a brief flow example of a login form submission.

⇒ @Controller: Marks the class as a controller that handles web requests.

@RequestMapping: Maps a specific URL (like /login) to a method inside a controller.

Model: A container used to pass data from the controller to the view.

In Spring MVC, when a browser sends an HTTP request, the framework follows the Model-View-Controller (MVC) design pattern to handle and respond to it in a structured way.

Request Handling Flow :-

① The request first goes to the ~~dispatcher~~ DispatcherServlet (the front controller).

② It looks for the correct method to handle the request using annotation like @RequestMapping.

③ The method is inside a class marked with @Controller, which acts as the controller in MVC.

④ Business logic is executed and necessary data is added to a Model object.

⑤ Finally the view is selected and returned with the model data for display.

Example: (Login form submission)

1. User submits a form to /login.
2. Spring calls the controller method:

```
@Controller
public class LoginController {
    @RequestMapping("/login")
    public String login(@RequestParam String username,
                        Model model) {
        if (username.equals("admin")) {
            model.addAttribute("message", "Login Successful");
        } else {
            model.addAttribute("message", "Login Failed");
        }
        return "login";
    }
```

21] Spring MVC uses the DispatcherServlet as a ~~front~~ controllen. Describe its role in the request processing workflow. How does it interact with view resolvers and handler mappings?

⇒ In Spring MVC, the ~~dispatcher~~ DispatcherServlet acts as the front controller, which means it is the central point for receiving all HTTP requests in a web application.

Role of DispatcherServlet in Request Workflow:-

① Receives Request: The DispatcherServlet receives the request from the browser.

② Finds Handler: It consults Handle Mapping to find the correct @controllen method based on the URL.

③ Calls Controller: It calls the mapped method in the controller to execute business logic.

④ Gets view Name: The controller method returns a logical view name.

⑤ Resolves View: The dispatcherServlet uses a View $ Resolver to map the logical name to an actual view file.

(VI) Renders View: The view is rendered with the data from the Model, and the final HTML is sent back to the browser.

How DispatchServlet Interacts with Handler Mappings and View Resolves :-

Handler Mapping: DispatcherServlet uses it to find the matching @Controller method based on the request URL.

View Resolver: After the Controller returns a view name, DispatcherServlet uses the view resolver to map that name to an actual view file.

Lab 6

22] How does PreparedStatement improve performance and security over Statement in JDBC? Write a short example to insert a record into a MYSQL table using PreparedStament.

⇒ • Performance: PreparedStatement precompiles the SQL query once and reuses it with different parameters. This makes it faster than Statement, especially for repeated operations.

• Security: It prevents SQL injection by safely binding user input as parameters instead of inserting raw strings into the query.

Example: (Insert Record Using Prepared Statement)

```
import java.sql.*;
public class InsertExample{
    public static void main(String args[]){
        try{
        Connection con = DriverManager.getConnection(
          "jdbc:mysql://localhost:3306/testdb","root","password");
          String query = "INSERT INTO
                students(name, email) VALUES(?,?)";
        PreparedStatement pst = con.prepareStatement(query);
          pst.setString(1, "Alice");
          pst.setString(2, "alice@example.com");
           int rows = pst.executeUpdate();
           System.out.println(rows + "record inserted.
              con.close();
              } catch(Exception e){
                  e.printStackTrace();
              }}}
```

Lab 7

23] What is ResultSet in JDBC and how is it used to retrieve data from a MySQL database? Briefly explain the use of next(), getString(), and getInt() methods with an example.

⇒ In JDBC, a ResultSet is an object that holds the result of a SQL SELECT query. It acts like a table in memory and allows you to retrieve and process rows one by one.

To retrieve data :-

① A SQL SELECT query is executed using Statement or PreparedStatement.

② The result is stored in a ResultSet object.

③ The next() method is used to move through each row.

④ Methods like getInt(), getString() etc are used to read column values from the current row.

Example:

```
ResultSet rs = stmt.executeQuery("SELECT id,
                    name From students");
while(rs.next()){
    int id = rs.getInt("id");
    String name = rs.getString("name");
    System.out.println("ID:" + id + ", Name:"
                        +name);
}
```

24] How does JPA manage the mapping between Java objects and relational tables? Explain with an example using @Entity, @Id, @GeneratedValue annotations. Discuss the advantage of using JPA over raw JDBC.

⇒ JPA (Java Persistence API) is a standard for mapping Java objects to relational database tables. It simplifies database operations by allowing developers to work with Java objects instead of writing raw SQL queries.

Mapping works:-

(i) @Entity: Marks a Java class as a database entity.

(ii) @Id: Marks a field as the primary key.

(iii) @GeneratedValue: Automatically generates the primary key value.

JPA (Java Persistence API) manages the mapping between Java objects and relational database tables using annotations. Each Java class is treated as Entity.

Example:

```
import jakarta.persistence.*;
@Entity
public class Student{
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;
 private String name;
 private String email;
 }
```

Here, the Student class will be mapped to a table named student. The id field is the primary key, and its value will be automatically generated. The name and email fields will be mapped to columns in the table.

- Advantages of JPA over JDBC:-

(i) Less Code: JPA reduces boilerplate (no need to write SQL for basic operations)

(ii) Object Oriented: Works directly with Java objects instead of rows and columns

(iii) Automatic Mapping: Maps classes to tables using annotations like @Entity.

(iv) Built-in Caching: Improves performance with automatic caching.

(v) Database Independent: Easier to switch databases without changing much code.

25] Describe the difference between the EntityManager's persist(), merge(), and remove() operations. When would you use each method in a typical database transaction scenario?

⇒ Difference between persist(), merge() and remove() in JPA is as follows:-

Method	Purpose	When to use
persist(obj)	Adds a new entity to the database.	When you want to insert a new record.
merge(obj)	Updates an existing entity.	When you want to update an existing record or reattach a detached object.
remove(obj)	Deletes an entity from the database.	When you want to delete a record

Usage in Transaction Scenario:-

- persist () → Use to insert a new student.

  Example: Student s = new Student ("Sup", "sup@mail.com");
  entityManager.persist (s);

- merge () → Use to update an existing student.

  Example: s.setName ("Sup Updated");
  entityManager.merge (s);

- remove (s) → Use to delete a student.

  Example: entityManage.remove (s)

All three methods should be used within a transaction, typically managed by @Transactional or EntityTransaction

Lab-8:

26] Design a simple CRUD application using Sprint Boot and MySQL to manage student records. Describe how each operation (Create, Read, Update, Delete) would be implemented using a reposity interface.

⇒

Goal: Manage student records with fields like id, name, email and course.

Technologies Used:

- Java
- Spring Boot
- Spring Data JPA
- MySQL

1] Entity class : Student.java

```
import jakarta.persistence.*;
@Entity
Public class Student{
    @Id
    @GeneratedValue (strategy = GenerationType.IDENTITY)
        private Long id;
        private String name;
        private String email;
        private String course;
    }
```

2] Repository Interface: StudentRepository.java

```
import org.springframework.data.jpa.repository.JpaRepository;
public interface StudentRepository extends JpaRepository<Student, Long>
{
}
```

3] Service Layer (StudentService.java)

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

@Service
public class StudentService{
@Autowired
   private StudentRepository repository;

//Create
Public Student saveStudent (Student student){
        return repository.save(student);
    }
// Read all
public List<Student> getAllStudents(){
    return repository.findAll();
}
```

```
public Optional<Student> getStudentById(Long id){
  return repository.findById(id);
}

public Student updateStudent(Long id, Student updateData){
    Student student = repository.findById(id).orElseThrow();
     student.setName(updatedData.getName());
      student.setCourse(updatedData.getCourse());
       return repository.save(student);
        }
public void deleteStudent(Long id){
        repository.deleteById(id);
      }}
```

4] @Controller (StudentController.java)

⇒

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/students")
public class StudentController {
    @Autowired
     private StudentService service;
```

```
@PostMapping
Public Student createStudent (@RequestBody Student student){
    return service.saveStudent (student);
  }
@GetMapping
public List <Student> getAllStudents (){
    return service.getAllStudents();
  }
@GetMapping ("{id}")
public Optional <Student> getStudent (@PathVariable Long id){
    return service.getStudentById (id);
  }
@PutMapping ("/{id}")
public Student updateStudent (@PathVariable Long id,
                              @RequestBody Student Updated)
  return service.updateStudent (id, updated);
}
@DeleteMapping ("/{id}")
public String deleteStudent (@PathVariable Long id){
  service.deleteStudent (id);
  return "Student deleted with id:" + id;
}
```

Explanation of how each CRUD operation (Create, Read, Update, Delete) is implemented using a repository interface in Spring Boot.

① Create (insert Data)

- Method: save (entity)
- Description: Saves a new entity into the database.

Example:

```
Student student = new Student ("Sup",
                  "Sup@gmail.com", "ICT");

studentRePository.save (student);
```

② Read (Fetch Data)

- Method: findAll(), findById(id)
- Description: Fetches all records or a specific record by Id

Example:

```
List <Student> list = studentRepository.findAll();
Optional<Student> student = studentRepository
                            .findById (1L);
```

(III) Update

- Method : save (entity)
- Description: Updates an existing record by setting the ID and modified data.

Example:

```
Student student = studentRepository.findById(1L).get();
student.setCourse("Data Science");
studentRepository.save(student);
```

(IV) Delete

- Method: deleteById (id)
- Description: Deletes a record using its ID.

Example:

```
studentRepository.deleteById(1L);
```

Lab - 9:

27] How does Spring Boot simplify the development of RESTful services? Describe how to implement a REST Controller using @RestController @GetMapping and @PostMapping including JSON data handling.

⇒ The process how Spring Boot simplifies RESTFUL Service Development:-

- Auto Configuration: Spring Boot auto-configures REST APIs using built-in support for JSON, Tomcat server, etc.
- Reduced Boilerplate:- No need to write XML or complex or complex setup.
- Embedded Server:- Comes with an embedded server (Like Tomcat) to run immediately.
- Built-in JSON Support: Automatically converts Java objects to JSON and vice versa using Jackson.

Rest Controller Implementation :-

① Creating a Model class (Student.java)

```
public class Student{
    private String name;
    private String course;
}
```

② Create a REST Controller (StudentController.java)

```
import org.springframework.web.bind.annotation.*;
import java.util.*;
@RestController
@RequestMapping("/student")
public class StudentController {
    List<Student> studentList = new ArrayList<>();
// Get : Fetch All students
@GetMapping
    public List<Student> getStudents(){
        return studentList;
    }

@PostMapping
public String addStudent(@RequestBody Student student){
    studentList.add(student);
    return "Student added successfully!";
}
}
```

JSON Example:

```
{ "name" : "Sup",
  "course" : "ICT"
}
```

29]

How does Maven manage dependencies and build lifecycle in a Spring Boot project? Explain the structure of a typical pom.xml and how the Spring Boot starter dependencies simply development?

⇒

Dependency Management:-

- Maven uses a file called pom.xml (project object model) to manage all required libraries.
- It automatically downloads libraries from the central repository.
- Developers just declare the dependency - Maven handles the rest.

Build Lifecycle:-

- Maven uses phases like:
  - compile : compiles the source code
  - test : Runs unit tests
  - package : Creates a .jar or .war file

- install: ~~install~~ adds the Project to local repo.
- deploy: Deploys to remote server.

Structure of a Typical pom.xml

```
<project xmlns = "http://maven.apache.org/pom/4.0.0"...>
    <modelVersion>4.0.0</modelVersion>

      <groupId> com.example </groupId>
      <artifactId> student-crud </artifactId>
      <version> 0.0.1-SNAPSHOT </version>

  <parent>
    <groupId> org.springframework.boot </groupId>
    <artifactId> spring-boot-starter-parent </artifactId>
     <version> 3.2.0 </version>
     </parent>

  <dependencies>
    <dependency>
     <groupId> org.springframework.boot </groupId>
     <artifactId> spring-boot-starter-web </artifactId>
     </dependency>

     <dependency>
     <groupId> mysql </groupId>
      <artifactId> mysql-connector-java </artifactId>
      </dependency>
```

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa
</artifactId>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

- How the Spring Boot Starter Dependencies Help:-

Starter Name	What It does
Spring-boot-starter-web	Adds REST APIs, Tomcat & JSON support
Spring-boot-starter-data-jpa	Adds Hibernate + JPA for database access
Spring-boot-starter-test	Adds JUNIT & Mockito for testing

30] Compare Maven and Gradles as build tools in Spring Boot projects. Discuss their syntax, performance, and dependency handling with an example of a ~~build.gradle~~ build.gradle. Configuration for a simple REST API.

⇒ Comparison of Maven Vs Gradle:-

Feature	Maven	Gradle
Syntax	XML (pom.xml)	Groovy/Kotlin (build.gradle)
Readability	Verbose, structured	concise, flexible
Performance	slower	Faster
Dependency	<dependency> blocks (XML format)	implementation/compileOnly
Build speed	slower on large projects	Generally faster.

Maven example:

xml

```
<dependency>
<groupId> org.springframework.boot </groupId>
<artifactId> spring-boot-starter-web </artifactId>
</dependency>
```

Gradle Example : build.gradle for REST API

groovy

```
plugins {
  id 'org.springframework.boot' version '3.2.0'
  id 'io.spring.dependency-management' version '1.1.0'
  id 'java'
}
group = 'com.example'
version = '0.0.1-SNAPSHOT'
sourceCompatibility = '17'
repositories {
    mavenCentral()
}
dependencies {
implementation 'org.springframework.boot:spring-boot-starter-web'

implementation 'org.springframework.boot:spring-boot-starter-data-jpa'

implementation 'mysql:mysql-connector-java:8.0.33'
testImplementation 'org.springframework.boot:spring-boot-starter-test'
}
```

Lab-10:

32] Demonstrate the project you developed with the important codes and Graphical User Interface.

⇒ I have developed a "Career ~~Counsib~~ Counselor" Web application using HTML, CSS and JavaScript. This project helps the users discover suitable career options based on their selected skills.

Project Overview:-

- Project Title: Career Counselor.
- Technology Used: HTML, CSS, JavaScript
- Execution: Client-Side only

Features Implemented:-

1. Collects user details :- Name, Age, Gender
2. Shows a list of skill checkboxes.
3. Suggests multiple jobs for each skill.
4. Displays a realistic matching percentage (60-86%) for each job.
5. Clean & Responsive GUI.

Important Code ~~Snip~~ Snippets:-

1. Skill to Job Mapping:-

```
const skillToJobs = {
```

develop: ['Software Engineer', 'Web Developer', 'Mobi
Developer'],
cp: ['Software Engineer', 'Competitive Programming Coa
ml: ['ML Engineer', 'AI Researcher'],
};

2] Job Match Scoring Logic :-

~~const~~
const JobSkillStrength = {
'Software Engineer': { ~~develop: 'strong', cp: 'me~~
develop: 'medium' }
'Web Developer': { develop: 'strong' },
. . . . . };
const skillWeight = { strong: 13, medium: 8 };
Object.entries(JobSkillStrength).forEach(([job, skillM
=> { let rawScore = 0;
selectedSkills.forEach(skill => {
if (skillsMap[skill]) rawScore += skillWeight
[skillsMap[skill]]; });
if (rawScore > 0) {
const percent = Math.min(Math.round(60 + rawScor
86);

```
jobScore[job] = percent;
}});
```

3] Result Display

```
<div class="job-item">
<div> Software Engineer </div>
<div class="job-score"> 86% match </div>
</div>
```

Graphical User Interface (GUI):-

- Form Section:-
  - White box with rounded corners and shadow.
  - Inputs for Name, Age, Gender
  - Grid of skill checkboxes.
- Button: Blue "Get Suggestions" button centered at the bottom.
- Result Section:
  - Each job shown in a card
    - Job emoji + name on the left
    - percentage badge on right.

Background: Gradient of stateblue and mediumpurple

Cards: clean, responsive, user-friendly layout.